



Experiment – 5

Name: Renulasya

UID: 24BAI70210

Section: 24AIT_NTTP2[A]

Branch/Semester: CSE –AIML(CS221) /5

Subject Code: 24CSP-337

Subject Name: Full Stack – II

Date of Performance:

Submitted to: Ms. Lovleen Kaur (E20454)

EXPERIMENT – 5.1

1. AIM: To design and implement RESTful APIs using Spring Boot with proper validation, standardized responses, and scalable architecture.

2. OBJECTIVE:

1. To understand REST API design principles.
2. To implement CRUD operations using Spring Boot.
3. To maintain consistent request and response structures.
4. To validate incoming data using Bean Validation.
5. To enable secure communication between frontend and backend using CORS.

3. SOFTWARE REQUIREMENTS:

1. Java JDK 17 or above
2. Spring Boot
3. Visual Studio Code / IntelliJ / Eclipse
4. Maven
5. Web Browser
6. HTML, CSS and JavaScript for frontend

4. THEORY

REST (**R**epresentational **S**tate **T**ransfer) is an architectural style used to develop scalable and loosely coupled web services. In REST, data is treated as **resources** and each resource is accessed using a unique URL.

REST APIs commonly use HTTP methods:

- **GET** – Retrieve data
- **POST** – Create new data
- **PUT** – Update existing data
- **DELETE** – Delete data

Spring Boot simplifies REST API development using a layered architecture:

Controller → Service → Repository → Database

The **Controller** handles HTTP requests, the **Service** contains business logic, and the **Repository** communicates with the database.

Bean Validation is used to validate incoming data using annotations such as `@NotBlank`, `@NotNull`, and `@Size`. This prevents invalid data from entering the system.

CORS (Cross-Origin Resource Sharing) allows a frontend application running on one origin to securely communicate with a backend running on another origin.

A well-designed REST API should provide consistent URLs, stateless communication, proper HTTP methods, validation, and predictable responses.

5. IMPLEMENTATION/PROCEDURE

1. Create a Spring Boot project using Java and Maven.
2. Configure the required Spring Boot dependencies.
3. Create the project package structure.
4. Create the **Model** class to represent post data.
5. Create the **Repository** layer to handle data storage.
6. Create the **Service** layer for business logic.
7. Create the **Controller** layer to handle API requests.
8. Implement CRUD operations using GET, POST, PUT and DELETE.
9. Apply validation annotations such as `@NotBlank` and `@Size`.
10. Configure CORS to allow frontend-backend communication.
11. Create a simple frontend using HTML, CSS and JavaScript.
12. Connect the frontend to the REST API using `fetch()`.
13. Run the Spring Boot application on port 8080.
14. Test creating, displaying, updating and deleting posts through the browser.

6. CODE

Controller

```
package com.example.postapi.controller;
```

```
import com.example.postapi.model.Post;
```

```
import com.example.postapi.service.PostService; import jakarta.validation.Valid;
```

```
import org.springframework.http.HttpStatus; import org.springframework.http.ResponseEntity; import  
org.springframework.web.bind.annotation.*;
```

```
import java.util.List;
```

```
@RestController @RequestMapping("/api/posts") @CrossOrigin(origins = "http://localhost:3000") public class  
PostController {  
    private final PostService postService;
```

```
    public PostController(PostService postService) { this.postService = postService;  
    }  
    ,
```

```
    // CREATE
```

```
    @PostMapping
```

```
    public ResponseEntity<Post> createPost( @Valid @RequestBody Post post) {
```

```
        return new ResponseEntity<>( postService.createPost(post), HttpStatus.CREATED  
        );  
    }
```

```
    // READ - All @GetMapping
```

```
    public ResponseEntity<List<Post>> getAllPosts()
```

Model

```
package com.example.postapi.model;
```

```
import jakarta.persistence.Entity;  
import jakarta.persistence.GeneratedValue;  
import jakarta.persistence.GenerationType;  
import jakarta.persistence.Id;  
import jakarta.validation.constraints.NotBlank;  
import jakarta.validation.constraints.Size;
```

```
@Entity
```

```
public class Post {
```

```
    @Id
```

```
    @GeneratedValue(strategy =  
GenerationType.IDENTITY)  
    private Long id;
```

```
    @NotBlank(message = "Title cannot be empty")
```

```
    @Size(min = 3, max = 100, message = "Title must  
be between 3 and 100 characters")  
    private String title;
```

```
    @NotBlank(message = "Content cannot be  
empty")  
    private String content;
```

```
    private String scheduledDate;
```

```
// Default constructor
```

```
public Post() {  
}
```

```
// Constructor
```

```
public Post(String title, String content, String  
scheduledDate) {  
    this.title = title;  
    this.content = content;  
    this.scheduledDate = scheduledDate;  
}
```

```
// Get ID
```

```
public Long getId() {  
    return id;  
}
```

```
// Title
```

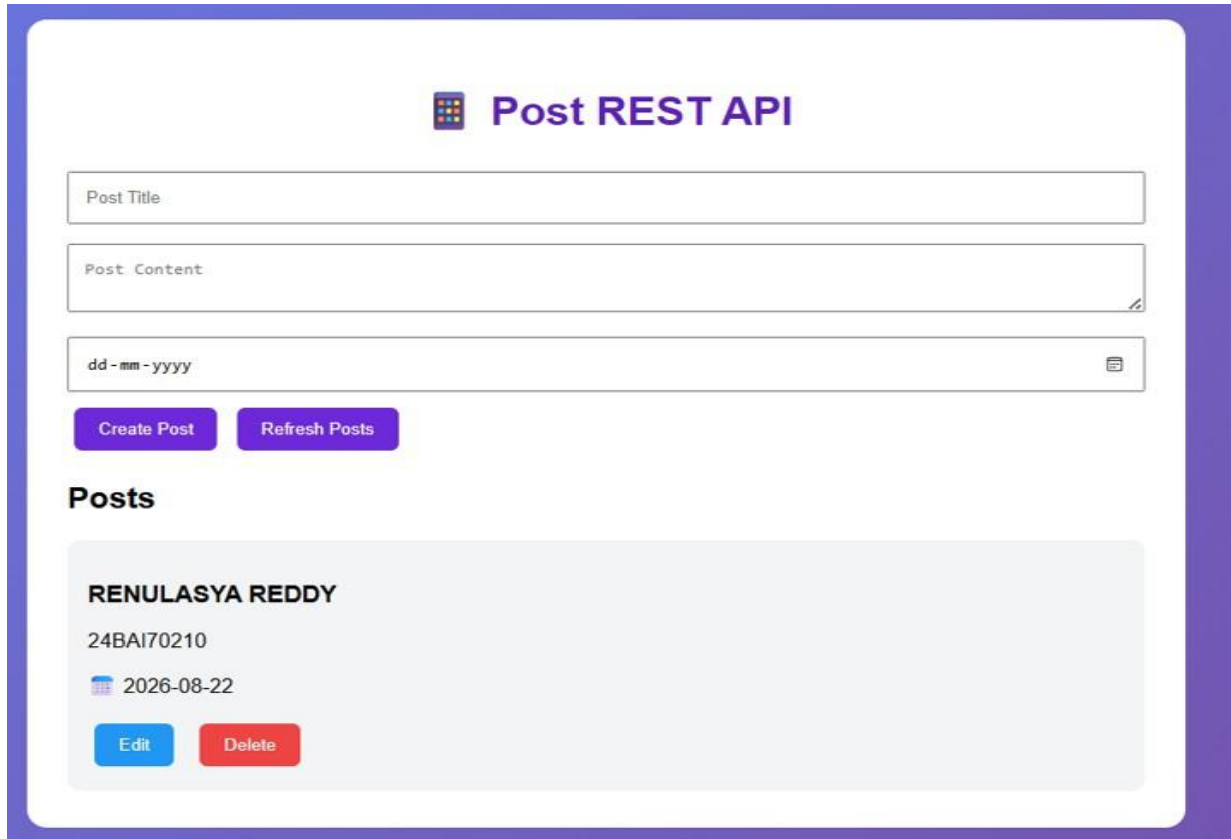
```
public String getTitle() {  
    return title;  
}
```

```
public void setTitle(String title) {  
    this.title = title;  
}
```

```
// Content
```

```
public String getContent() {  
    return content;  
}
```

7. OUTPUT



The screenshot displays a web application titled "Post REST API". It features a form with three input fields: "Post Title", "Post Content", and a date field labeled "dd-mm-yyyy". Below the form are two buttons: "Create Post" and "Refresh Posts". Underneath the form, there is a section titled "Posts" which contains a list of posts. The first post shown is by "RENULASYA REDDY" with ID "24BAI70210" and date "2026-08-22". At the bottom of this post entry are two buttons: "Edit" and "Delete".

Fig 1.1: Post REST API – CRUD Operations Interface

8. LEARNING OUTCOME

- Understand the basic principles of RESTful APIs.
- Develop CRUD APIs using Spring Boot.
- Understand the Controller, Service and Repository architecture.
- Apply validation to API requests.
- Understand and configure CORS.
- Connect a frontend application with a Spring Boot backend.
- Develop a scalable and maintainable backend application.

EXPERIMENT – 5.2

1. **AIM:** To implement global exception handling and structured logging for building robust and observable backend systems.

2. **OBJECTIVE:**

1. To handle exceptions centrally using `@ControllerAdvice`.
2. To implement logging mechanisms for request tracking.
3. To use correlation IDs for tracing requests.
4. To improve system observability and debugging.

3. **SOFTWARE REQUIREMENTS:**

1. Java JDK 17+
2. Spring Boot
3. Visual Studio Code / IntelliJ / Eclipse
4. Maven
5. SLF4J / Logback
6. Web Browser

4. **THEORY**

As In a backend application, errors can occur during request processing. Handling every exception separately in each controller can make the code lengthy and difficult to maintain.

Spring Boot provides **global exception handling** using `@RestControllerAdvice` or `@ControllerAdvice`. It allows exceptions from different controllers to be handled at one central location and provides a consistent error response.

Logging is used to record important information about application execution. It helps developers understand what happened during a request and makes debugging easier.

A **Filter** can intercept incoming HTTP requests before they reach the controller. It can be used to record:

- Request URL
- HTTP method
- Response status
- Execution time
- Correlation ID

A **Correlation ID** is a unique ID assigned to each request. It helps identify and trace a particular request through the application logs.

A **custom exception** such as `NotFoundException` can be created for specific errors. When the exception occurs, the global exception handler catches it and returns a structured response containing information such as status, message and timestamp.

Thus, exception handling and logging make backend applications more **reliable, maintainable, observable and easier to debug**.

5. IMPLEMENTATION/PROCEDURE

1. Create a Spring Boot project using Java and Maven.
2. Create the required packages:
 - controller → exception → filter → model
3. Create the Post model class.
4. Create a custom PostNotFoundException class.
5. Create GlobalExceptionHandler using @RestControllerAdvice.
6. Handle PostNotFoundException using @ExceptionHandler.
7. Return a structured error response containing:
 - Timestamp
 - Status
 - Error
 - Message
8. Create a LoggingFilter to intercept HTTP requests.
9. Generate a unique correlation ID for every request.
10. Log the request URL and HTTP method.
11. Log the response status and execution time.
12. Create REST endpoints for retrieving posts.
13. Run the Spring Boot application on port 8080.
14. Open the frontend in the browser.
15. Test a valid request such as: **GET /api/posts**
16. Test an invalid request such as: **GET /api/posts/99999**
17. Observe the structured 404 Not Found response and corresponding logs in the VS Code terminal.

6. CODE

Model

```
package com.example.exception_api;
import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class ExceptionApiApplication {

    public static void main(String[] args) {
        SpringApplication.run(ExceptionApiApplication.c
lass, args);
    }

}
```

Exception

```
package com.example.exception_api.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(PostNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public Map<String, Object> handlePostNotFound(
        PostNotFoundException ex) {

        Map<String, Object> error = new HashMap<>();

        error.put("timestamp", LocalDateTime.now());
        error.put("status", 404);
        error.put("error", "Not Found");
        error.put("message", ex.getMessage());

        return error;
    }
}
```

Controller

```
package com.example.exception_api.controller;

import com.example.exception_api.exception.PostNotFoundException;
import com.example.exception_api.model.Post;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import org.springframework.web.bind.annotation.*;

import java.util.ArrayList;
import java.util.List;

@RestController
@RequestMapping("/api/posts")
public class PostController {

    private static final Logger logger =
        LoggerFactory.getLogger(PostController.class);

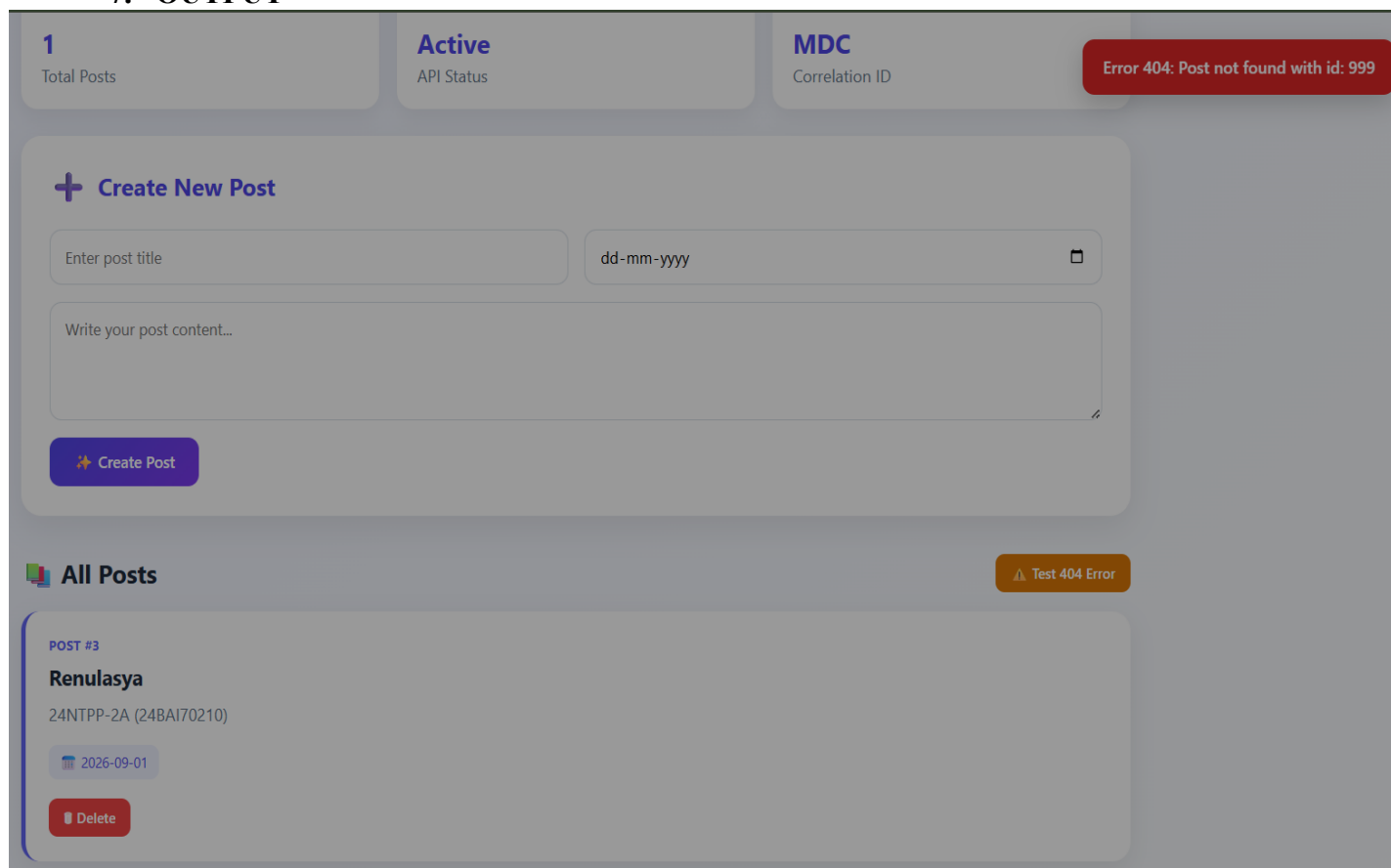
    private final List<Post> posts = new ArrayList<>();

    public PostController() {

        posts.add(new Post(1L, "Renu Lasya"));
        posts.add(new Post(2L, "24BAI70210"));
    }

    // GET ALL POSTS
```

7. OUTPUT



The screenshot displays a web application interface for managing posts. At the top, there are three tabs: '1 Total Posts', 'Active API Status', and 'MDC Correlation ID'. A red error message 'Error 404: Post not found with id: 999' is shown in the top right corner. Below the tabs is a 'Create New Post' section with a form containing a title input field, a date input field (dd-mm-yyyy), and a content text area. A 'Create Post' button is located below the form. Underneath the form is an 'All Posts' section showing a list of posts. The first post is 'POST #3' by 'Renulasya' with ID '24NTPP-2A (24BAI70210)' and a date of '2026-09-01'. A 'Delete' button is present next to the post. A 'Test 404 Error' button is also visible in the top right of the 'All Posts' section.

Fig 1.1: Successful API Request with Structured Logging

8. LEARNING OUTCOME

- Understood global exception handling in Spring Boot.
- Create and handle custom exceptions.
- Use `@RestControllerAdvice` for centralized error handling.
- Implement request and response logging.
- Understand the purpose of correlation IDs.
- Trace requests using application logs.
- Build more robust and observable backend applications.